

Definizione: Una DCFG A è una CFG tale che, ogni stringa valida ha un "handle forzato".

Se $v = xhy$ è una stringa valida, $(h, T \rightarrow h)$ è un "handle forzato" se questo handle è l'unico handle che si può applicare in ogni stringa valida della forma xhu , dove $u \in \Sigma^*$, u è una qualunque stringa di terminali!

Un handle forzato è un handle che, è sempre lo stesso, a prescindere da qual'è la stringa valida di terminali che segue l'handle.

Una grammatica è deterministica se ogni stringa valida ha un handle forzato !!!

Non posso provare con tutte le stringhe e vedere se sono forzate nelle riduzioni, NO!

Mi serve un modo per capire se una grammatica è deterministica oppure NO!

Questo modo esiste e si chiama:

DK-TEST (deterministic K - test)

• CONSISTE NEL COSTRUIRE UN CERTO DFA e guardarlo!

PER COSTRUIRE IL DFA SI PASSA DA UNA MACCHINA K che è NON DETERMINISTICA e per arrivare a questa macchina "NON DETERMINISTICA" si passa in un'altra macchina "NON DETERMINISTICA" chiamata J .

Poi alla fine noi possiamo costruire direttamente l'automa DK a partire dalla grammatica!

(questa è la tecnica di Donald Knuth).

NOI COSTRUIAMO L'AUTOMA DFA DK PER VEDERE SE È DETERMINISTICA O NO.

Per ogni CFG G possiamo costruire un DFA DK. Questo DFA DK accetta un input z se e solo se:

1) z è il prefisso di una stringa valida v : $v = z\gamma$

2) z termina con un handle di v .

Nell'esempio di prima, il mio z sarebbe: con $uv = xhy$
 $\downarrow$
 z

Quindi, Passandogli questa stringa, il mio dfa la accetterebbe!

Il dfa riconosce gli handle della grammatica G .

Quando arrivo ad uno stato di accettazione di questo automa, io riesco a riconoscere quali regole si applicano per arrivare a ridurre la stringa!

Come si costruisce quest' automa?

cominciamo dalla macchina J .

io ho una grammatica G con delle regole: $A \rightarrow u$
 $B \rightarrow v$

La mia macchina J è un NFA che accetta fondamentalmente ogni stringa di input in cui, la parte destra è una regola della grammatica!

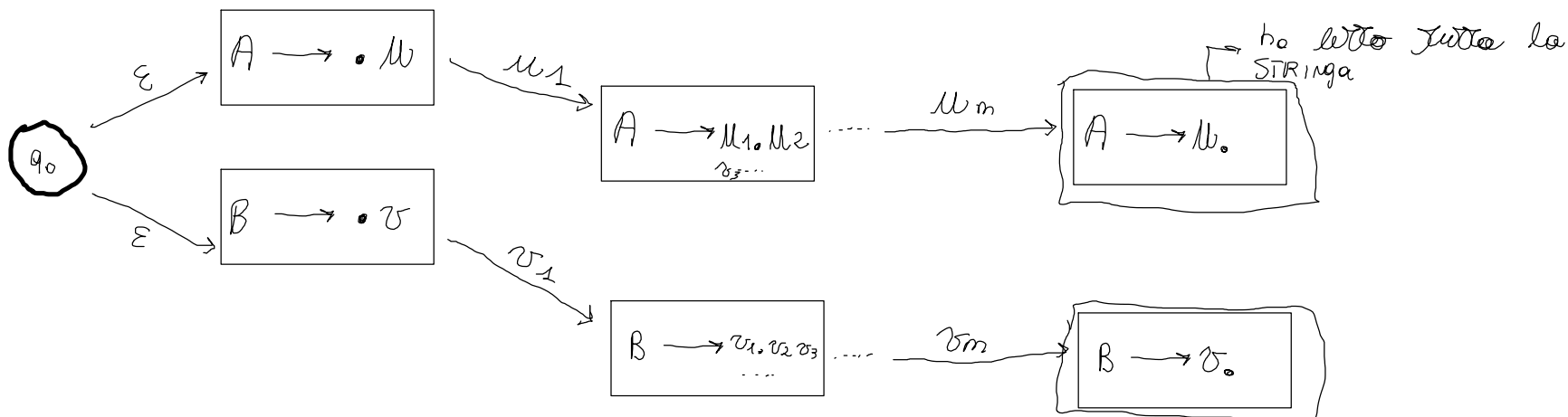
$J(w)$ accetta se e soltanto se w parte destra di una regola:

Come lo costruiamo? Prendo tutte le regole e le rendo uno stato di J .

Però questa regola va chiamata **dotted rule** perché ci aggiungo un puntino.

se $w = w_1 w_2 \dots w_3 w_n$
se $v = v_1 v_2 \dots v_3 v_n$

IL PUNTINO rappresenta dove sono arrivati a leggere l'INPUT seguendo questa regola!

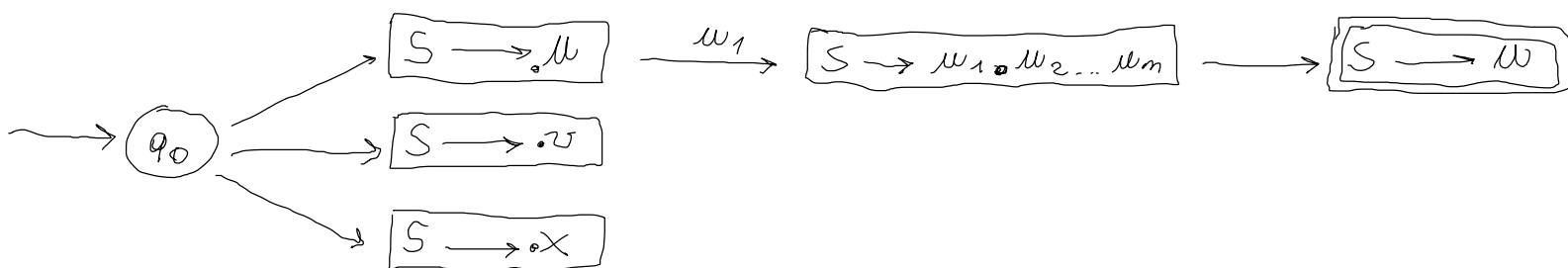


Tutti gli stati che hanno il dot a destra, sono stati di accettazione!

Quindi c'è uno stato iniziale q_0 e non deterministicamente indovino la regola e segue tutte le transizioni un carattere alla volta, fino ad accettare.

Costruiamo la macchina K , a partire dalla J

Questa macchina K è un NFA che, a partire da uno stato iniziale va soltanto sulle regole che espongono dalla variabile iniziale della grammatica!



$S \rightarrow$ ogni volta che la mia regola espande in una stringa lunga faccio lo stesso procedimento.

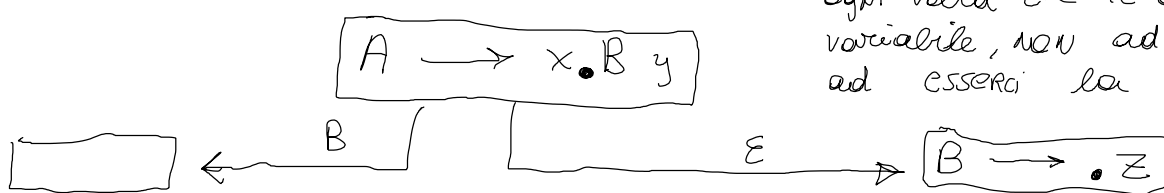
se $w = w_1 w_2 w_3 \dots w_m$

Se ho:

$A \rightarrow x B y$

$B \rightarrow z$

io prendo gli stati e li collego con frecce ϵ :



ogni volta che il dot si trova davanti una variabile, non ad un simbolo terminale, oltre ad esserci la freccia B e quindi procedere con la catena verso il dot alla fine, c'è una freccia ϵ che mi riporta allo stato corrispondente alla regola iniziale $B \rightarrow . z$, per qualunque regola di B

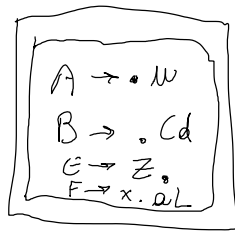
La macchina DK significa prendere K e renderla deterministica. **ogni NFA ha il suo**

DFA deterministico

Per gli esercizi non serve tutto questo!

Una volta che ho il dk, com'è che decido se la grammatica di partenza era deterministica o no?

devo prendere tutti gli stati di accettazione, cioè tutti gli stati che contengono almeno una regola completata!



quando dal NFA passo al DFA, dentro ci possono essere sia regole completate che non completate.

DK fallisce se esistono due regole completate nello stesso stato di accettazione e quindi la grammatica non è deterministica.

Cioè posso costruire una stringa valida con due handle.

DK fallisce se esiste una regola dove il dot precede un terminale

1. DK è OK hanno una sola regola completata e tutte le altre regole il dot precede una variabile e quindi la grammatica è deterministica, quindi l'handle è forzato